



Storm-Gauge Response Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 04-April-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

1 Test Results — Storm-Gauge Response Model (MKM-SG-001)

Tests run on **04 April 2026 at 14:48:33**.

Table 1: Test Results Summary — Storm-Gauge Response Model (MKM-SG-001)

Total Tests	41
Passed	41
Failed	0
Skipped	0
Duration	0.03s

Table 2: Detailed Test Results — Storm-Gauge Response Model

Test Description	Acceptance Criteria	Result	Time
Encodes datetime	"2026-01-01" in result	Pass	0.000s
Line 80: np.float32 is np.floating but NOT a Python float subclass in NumPy 2.x,	"2.5" in result	Pass	0.000s
Encodes numpy floating	"3.14" in result	Pass	0.000s
Encodes numpy integer	"42" in result	Pass	0.000s
Encodes numpy ndarray	"1.0" in result	Pass	0.000s
Fallback raises for unknown	—	Pass	0.000s
Close to thames high risk	result["FloodRiskScore"] >= 7; result["FloodRiskCategory"] in ["High", "Very High"]	Pass	0.000s
Far from thames lower risk	result["FloodRiskScore"] <= 6; result["FloodRiskCategory"] in ["Low", "Medium"]	Pass	0.000s
Flood stages ordering	alert < warning < severe	Pass	0.001s
Processing stats	stats["successful_gauges"] == 5; stats["failed_gauges"] == 0	Pass	0.001s
Lines 264-266: exception during json.dump is logged and re-raised.	—	Pass	0.000s
Line 245: end_time is set by the first generate(); the second generate() call	—	Pass	0.001s
Line 290: non-dict section_schema returns {}.	result == {}	Pass	0.000s
Line 294: 'type', 'options', 'description', 'values' keys are skipped.	skip not in result	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Lines 159-162: count i max_gauges → capped to max_gauges.	<code>len(result["data"]["flood_gauges"]) == max_available</code>	Pass	0.007s
Line 155-156: count i 1 raises ValueError.	—	Pass	0.000s
Lines 354-355: generate_gauges() convenience function.	<code>"data" in result; len(result["data"]["flood_gauges"]) == 3</code>	Pass	0.001s
Lines 212-215: exception in _generate_single_gauge increments failed_gauges.	<code>result["processing_stats"]["failed_gauges"] == 1</code>	Pass	0.000s
Lines 122-125: log() dispatches to correct logger method.	—	Pass	0.000s
Lines 309-327: missing Flood- Gauge/Header/Location/Flood- Stages are created.	<code>gauge_data["FloodGauge"]["Header"]["GaugeID"] == "GAUGE-t..."; gauge_data["FloodGauge"]["Location"]["GaugeLatitude"] == ...</code>	Pass	0.000s
Lines 99-100: verbose=False sets logger to WARNING.	<code>logging.getLogger("port.src.gauge.gauge") == login...</code>	Pass	0.000s
Output file exists	<code>result["file_path"].exists()</code>	Pass	0.001s
Output file is valid json	<code>"flood_gauges" in data</code>	Pass	0.002s
Output json has correct count	<code>len(data["flood_gauges"]) == 5</code>	Pass	0.004s
Lines 378-381: __main__ block calls generate_gauges and logs results.	<code>(tmp_path / 'gauge.json').exists()</code>	Pass	0.005s
Gauge id format	<code>re.match(r"^GAUGE-[a-f0-9]{8}\$", gid)</code>	Pass	0.001s
Gauge ids unique	<code>len(ids) == len(set(ids))</code>	Pass	0.001s
Generate correct count	<code>len(result["data"]["flood_gauges"]) == 5</code>	Pass	0.001s
Generate returns expected keys	<code>"data" in result; "file_path" in result (+1 more)</code>	Pass	0.003s
Locations have coordinates	<code>"lat" in loc; "lon" in loc (+1 more)</code>	Pass	0.001s
Date field returns date string	<code>isinstance(val, str); re.match(r"^\{d{4}-\{d{2}-\{d{2}\}\$", val)</code>	Pass	0.000s
Decimal field returns float	<code>isinstance(val, (int, float))</code>	Pass	0.000s
Gauge name includes area	<code>"TestArea" in val</code>	Pass	0.000s
Menu field returns valid option	<code>isinstance(val, str)</code>	Pass	0.000s
Text field returns string	<code>isinstance(val, str); len(val) > 0</code>	Pass	0.000s
Catchment name stored	<code>meta["catchment_name"] == "Thames"</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Flood stage ordering	<code>meta["flood.alert"] <</code> <code>meta["flood.warning"];</code> <code>meta["flood.warning"] <</code> <code>meta["severe_flood.warning"]</code> (+1 more)	Pass	0.000s
Gauge id format	<code>re.match(r"~GAUGE-[a-f0-9]{8}\$",</code> <code>meta["gauge_id"])</code>	Pass	0.000s
Historical high level range	<code>5.0 <= meta["historical_high_level"]</code> <code><= 8.0</code>	Pass	0.000s
Install date format	<code>re.match(r"~\{d{4}-\{d{2}-\{d{2}\$",</code> <code>meta["install_date"])</code>	Pass	0.000s
Returns dict with required keys	<code>key in meta</code>	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (26 tests)	D. Kelly
07-Mar-2026	1.1	23 test(s) added; 26 test(s) removed (total: 23)	D. Kelly
09-Mar-2026	1.2	40 test(s) added (total: 63)	D. Kelly
14-Mar-2026	1.3	14 test(s) added (total: 77)	D. Kelly
27-Mar-2026	1.4	4 test(s) added (total: 81)	D. Kelly
04-Apr-2026	1.5	40 test(s) removed (total: 41)	D. Kelly